ThetaGate

ThetaGate is an automatic options desk for paper trading. It sells defined-risk credit spreads when implied volatility is high compared to realized volatility. At all other times it opens no new position. It still manages and closes open positions.

Alpaca AI Trading Agents Hackathon. Track: Options Alpha Agents. September 2026.

STRUCTURE Put and call credit spreads	SIGNAL ATM IV / 20-day RV, 1.20 or more	MODE Paper trading only. No live path.
MODEL Featherless, then Grok	BROKER Alpaca: API, MCP, CLI	CODE `github.com/theaimlessfox-hackathons/alpaca-ai-trading-agents`

Many options agents choose a direction and accept unlimited risk.

They buy single options or sell uncovered premium. They also let the model send orders. One large price move can cause a large loss.

01 **Directional trades.** The agent buys calls or puts. A wrong direction gives a loss.

02 **Undefined risk.** An uncovered short option has no maximum loss. A large price move gives a large loss.

03 **The model sends the order.** If the model returns bad data, the agent can open a bad position.

ThetaGate approach: limit the risk on every structure. Do not let the model send an order. Python makes the final decision.

Sell defined-risk premium only when option prices are high compared to recent price movement.

Each cycle, Python compares the at-the-money implied volatility with the 20-day realized volatility. A ratio of 1.20 or more opens the gate.

01 **High volatility.** If IV / RV is 1.20 or more, the desk sells a vertical spread with 7 to 21 days to expiration. The short leg delta is 0.20 to 0.30. The long leg delta is 0.10 to 0.15.

02 **Low volatility.** If the ratio is below the gate, the desk opens no new position and sends no proposal to the model.

03 **Breakout.** A price range breakout stops new entries. Open positions stay under exit management. This rule is independent of volatility.

Regime, not IV rank. A live IV / RV ratio changes on the same day. A percentile rank changes many weeks after the volatility regime changes.

Defined risk at all times. The maximum loss is `(width − credit) × 100 × quantity`. Python knows this value before it builds the order. The limit is 2 percent of NAV.

The model proposes. Python decides. Only the execution module sends an order.

Each cycle has seven steps. The model sees steps 3 and 4 only. It never gets a tool to place, cancel, close, replace, or exercise an order.

01 **Discover.** Before the open, the desk ranks a 10 to 20 symbol watchlist from Alpaca activity, price moves, and news. It refreshes the list once after midday. A failed discovery gives an empty book, with no hidden fallback list.

02 **Regime.** Python gets price bars and an option chain for the DTE band, fills any missing delta and IV with the Black-Scholes method, then checks that IV / RV is 1.20 or more.

03 **Propose.** Featherless returns a two-leg `TradeProposal` from the contracts that passed. Invalid JSON: retry 3 times or fewer, then Grok, then stop.

04 **Critic.** A second model reviews the proposal. This review is advisory. The critic cannot approve the trade and cannot spend money.

05 **Risk engine.** Python recalculates the maximum loss, DTE, both deltas, spread width, and IV, and checks every book limit. A bad proposal becomes a stored veto.

06 **Execute.** The desk sends one multi-leg order at the natural credit. A structure changes to OPEN only after Alpaca reports `filled_qty` greater than 0.

07 **Reconcile and exit.** The desk monitors the marks. A take-profit, a stop, a regime change, or STOP starts one atomic close for the reconciled open quantity.

Five limits on the model. Code applies them, not the prompt.

Remove all text from the system prompt. The result does not change. The model still cannot reach the broker.

No order tools
The proposer and the critic get market data only. They never get `place_option_order`, cancel, close, replace, or exercise. Only the execution module sends an order.

No new contracts
The model selects from a list of real option-chain contracts. Each contract already passed the DTE, delta, and liquidity checks. The desk rejects any contract that is not on the list.

Invalid output gives no trade
If the JSON is not valid, the desk retries 3 times or fewer, then uses Grok, then stops the cycle. 15 of 372 cycles stopped here. None became a position.

Model numbers are not trusted
Python calculates the maximum loss, both deltas, the spread width, and the credit again from the live chain.

The critic cannot act
It returns `advisory_only: true` and `place_orders: false`. It can record an objection in the log. It cannot approve, size, or spend.

Prompt-independent by design. Each limit is a Python check or a tool that the model does not have. The prompt affects the quality of a proposal. It does not affect what the desk can execute.

A spread inside every band. The book limit stopped it.

Cycle log, session 2026-09-02. Both leg deltas were inside the set bands. The risk engine did not approve the trade.

INTC PUT CREDIT SPREAD exp 2026-09-11 · DTE 10

SHORT	put **84.0**	Δ −0.256	IV 0.53 · 1.20 / 1.29
LONG	put **80.0**	Δ −0.126	IV 0.56 · 0.53 / 0.55

width 4.00 · natural credit about 0.65 · defined maximum loss about 335 dollars (0.34 percent of NAV)

“IV / RV at 1.22 supports the sale of defined-risk premium. The 84/80 put spread keeps both deltas inside the set bands, DTE 10, with tight quotes on both legs.”

VETO

`open_count`. 3 structures are already open (book limit). The desk also applies a limit of 2 per underlying.

The check is independent. Delta, width, DTE, and maximum loss passed a separate Python calculation. The proposer does not see the book capacity check.

This is timing, not a permanent block. The same 84/80 structure traded later in the session when capacity became free. The desk records every veto.

High volatility, deltas in band, capacity free. Approved and sent to paper trading.

Cycle log, session 2026-09-01. The critic raised an objection. The objection did not stop the trade. The desk added it to the record.

SPY PUT CREDIT SPREAD exp 2026-09-08 · DTE 7 · IV / RV 1.55

SHORT	put **753**	Δ about −0.24
LONG	put **747**	Δ about −0.14

width 6.00 · net credit about 0.77 · defined maximum loss about 523 dollars (0.52 percent of NAV) · day limit order at the natural credit

“IV / RV at 1.55 supports the sale of defined-risk premium. The 7-day 753/747 put spread keeps both deltas inside the set bands. Put skew adds credit versus calls.”

APPROVED

The critic raised an advisory objection about risk, reward, and event risk (`place_orders: false`). Python then recalculated the maximum loss, deltas, width, and capacity and sent the order. Broker order `58a52f23`. State `PENDING_ENTRY`.

Natural credit limit. The entry price is the short bid minus the long ask, so an approved order can fill instead of resting at the mid price.

The state follows the fill. The structure stayed at `PENDING_ENTRY` until Alpaca reported a fill. The desk cancels an unfilled entry and voids the structure.

Deterministic risk gates. Python runs them again immediately before each order.

No model output reaches the broker before it passes every gate. The full list is in `config/settings.py`.

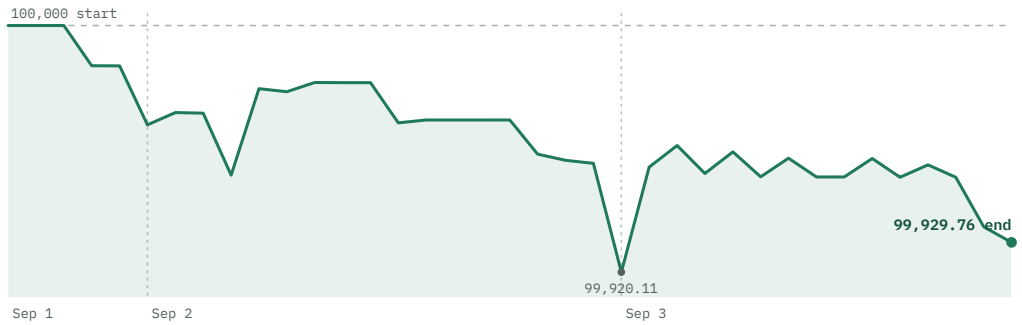
GATE	RULE
Risk per structure	Maximum loss `(width − credit) × 100 × quantity` is 2 percent of NAV or less. Python computes it independently.
Book capacity	`3 open structures or fewer. 2 per underlying or fewer.`
Deltas and DTE	`DTE 7 to 21. Short Δ 0.20 to 0.30. Long Δ 0.10 to 0.15.`
Regime	ATM IV / 20-day RV is `1.20` or more. If not, the desk sends no proposal and opens no new position.
Quote quality	Relative bid-ask spread 20 percent or less per leg. Positive net credit. Sane IV.
Equity halts	Minus 3 percent daily or minus 8 percent total: cancel unfilled orders, then close all positions.
Exits	Take-profit at 50 percent of credit. Stop at 2 times credit. Regime change confirmed twice. STOP.
Kill switch	`logs/KILL` stops new entries and closes the book. It stays open until no unfilled orders and no open positions remain.
Atomic close	One multi-leg debit for the reconciled open quantity, or the operation stops with no change. Never one leg at a time.

Model authority: proposer and critic only. The model never gets `place_option_order` or any order tool. `ALPACA_PAPER_TRADE` set to false is rejected.

372 cycles from 2026-09-01 to 2026-09-03. The system ran with no unresolved states.

Sandbox paper account PA3K…0J1 (full ID on the submission form). Options level 3.
Start 100,000 dollars, end 99,929.76 dollars. Values come from the SQLite ledger.

no trade: 300 (193 low IV / RV, 105 breakout)
no viable contracts: 20
invalid model JSON, no trade: 15
risk veto: 15
approved: 22 (7 dry run, 15 sent to paper)



equity: 100,000 to 99,929.76

peak drawdown 0.08 percent · daily halt 3 percent

372 cycles across 3 sessions	**81%** cycles with no trade. The desk waited for high volatility.
9 / 7 entries filled / closed with one atomic order	**0** NEEDS_REVIEW states, one-leg closes, and partial fills

The 3-session run confirms correct operation: the regime gate, the risk vetoes, the fill-linked states, the atomic closes, and the recovery of unfilled entries. Risk stayed well inside every halt limit. Judges score performance on the competition account.

Alpaca for every market action. Python for every decision.

Alpaca Trading API
Paper account, options level 3, 100,000 dollars. No supported live path.

Official Alpaca MCP
`alpaca-mcp-server`. Account, clock, price bars, option chain, news, and multi-leg `place_option_order`. Contracts use OCC symbols. A credit is a negative limit price.

Alpaca CLI
`alpaca account`. One read. This meets the CLI requirement.

Market data REST
The screener uses the most active list and the largest price moves. It also gets stock snapshots and Alpaca news.

Featherless
It returns a structured `TradeProposal`. It also runs the advisory critic. Grok is the backup when the JSON is not valid.

Black-Scholes method
Python calculates delta and IV from the mid quote when the data feed does not supply them. This is the normal path on this account.

Storage
An SQLite ledger and a JSONL decision log. Every veto, fill, and equity mark is available for query.

Streamlit desk on Railway
It runs with the live scheduler. The desk monitors positions and provides STOP. The desk does not place orders.

`github.com/theaimlessfox-hackathons/alpaca-ai-trading-agents` sandbox account **PA3K…OJ1** · competition account ID on the submission

Paper trading only. This is not investment advice.